Multi-Task Optimization over Networks of Tasks

Julian Hatzky[1], Thomas Bartz-Beielstein[2], A. E. Eiben[1], and Anil Yaman[1]

[1] Vrije Universiteit Amsterdam, Amsterdam, The Netherlands
{j.hatzky, a.e.eiben, a.yaman}@vu.nl
[2] TH Köln, Cologne, Germany
thomas.bartz-beielstein@th-koeln.de

Abstract. Multi-task optimization is a powerful approach for solving a large number of tasks in parallel. However, existing algorithms face distinct limitations: Population-based methods scale poorly and remain underexplored for large task sets. Approaches that do scale beyond a thousand tasks are mostly MAP-Elites variants and rely on a fixed, discretized archive that disregards the topology of the task space. We introduce MONET (Multi-Task Optimization over Networks of Tasks), a multi-task optimization algorithm that models the task space as a graph: tasks are nodes, and edges connect tasks in the task parameter space. This representation enables knowledge transfer between tasks and remains tractable for high-dimensional problems while exploiting the topology of the task space. MONET combines *social learning*, which generates candidates from neighboring nodes via crossover, with *individual learning*, which refines a node's own solution independently via mutation. We evaluate MONET on four domains (archery, arm, and cartpole with 5,000 tasks each; hexapod with 2,000 tasks) and show that it matches or exceeds the performance of existing MAP–Elites-based baselines across all four domains.

Keywords: Multi-task Optimization · Multi-Task MAP-Elites · Innovation Networks · Social Learning · Graphs

1 Introduction

To maintain reliable performance, real-world embodied agents must remain operational even when their configurations change. For example, in scenarios where a legged robot loses a limb [11,14] or a modular robot reconfigures itself to navigate a narrow passage [18,44,52], they are required to maintain their performance. In such cases, the ability to adapt an existing control strategy instead of relearning a new strategy for the new configuration from scratch can prove to be more efficient. It is possible to achieve this by employing the multi-task optimization approach [23], where each configuration variant (e.g. morphology) is treated as a distinct task, and solutions (e.g. controller) are pre-computed for all of them, so that when the configurations change, an appropriate controller is already available or is faster to adapt [12,38]. In other words, given a parameterized family of tasks, the multi-task optimization approach aims to find a high-quality solution for each task simultaneously and in parallel [23].

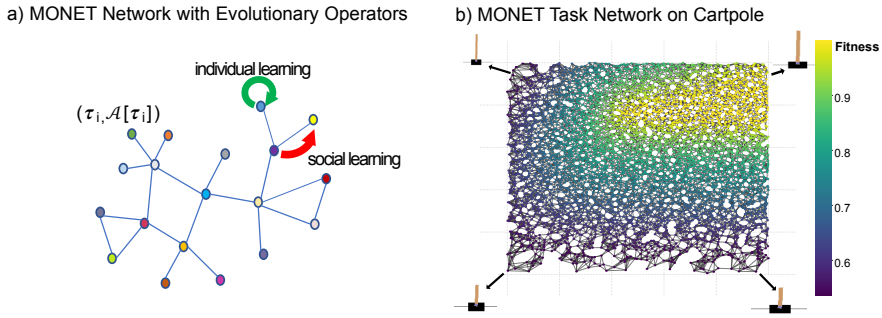


Fig. 1: Illustration of the MONET task network structure. Each node $V_i = \boldsymbol{\tau}_i$ represents a task, labelled by its current best solution $\mathcal{A}[\boldsymbol{\tau}_i] \in \Theta$. (a) Task network with the individual and social learning strategies that operate on it. (b) Task network and solution qualities at 500,000 evaluations (50% of the total).

Existing multi-task optimization algorithms face a fundamental scalability–structure tradeoff. Population-based multi-task evolutionary methods [29, 49] maintain a separate population per task and quickly become impractical as the number of tasks grows into the thousands and beyond. Quality-diversity (QD) algorithms, and MAP-Elites [14, 38] in particular, bypass this by replacing per-task populations with a single structured archive. MAP-Elites has been extended to multi-task settings by Multi-Task MAP-Elites (MT-ME) [39], which solves a fixed set of tasks via a globally shared archive, and Parametric-Task MAP-Elites (PT-ME) [4], which extends this to continuous task spaces. Both have demonstrated strong performance on problems with thousands of tasks.

However, these archive-based methods exhibit two intertwined limitations. First, the archive is a *fixed discretized grid* over the task or feature space: with c bins per dimension and a d-dimensional task space its size grows as $\mathcal{O}(c^d)$, so the representation becomes infeasible already for moderate d. Second, the archive is largely *topologically flat*: in MT-ME any solution can serve as a parent for any task. PT-ME partially addresses this through a local linear regression operator that exploits Delaunay triangulation [17] between tasks to calculate an adjacency matrix. Even so, this locality is confined to one of two variation paths, and the global Simulated Binary Crossover (SBX) operator [16] still draws parents without regard for task similarity. Neither method makes the task-space topology a first-class structural element that governs all recombination.

We introduce MONET (Multi-Task Optimization over Networks of Tasks), a multi-task optimization algorithm that resolves both limitations by replacing the flat archive with an explicit graph over tasks (Figure 1): each node in the vertex set $\mathcal{V}$ represents a task and stores its current best solution, while bidirectional edges in $\mathcal{E}$ connect tasks to enable knowledge transfer (e.g., through crossover or direct copy). Unlike a grid, this representation does not depend on a discretization of the task space and scales as $\mathcal{O}(|\mathcal{V}|\, k)$, where k is the number of neighbors per node, remaining tractable as the task space dimensionality

grows. Unlike a flat archive, the graph encodes topology directly: every node has a well-defined neighborhood that the search can exploit. The evolutionary process operates on this graph through two complementary mechanisms. *Individual learning* improves a solution in isolation through mutation, exploring the local fitness landscape of a single task. *Social learning* transfers information between neighboring nodes in variable ratios by selecting parent solutions from neighboring tasks and generating offspring through crossover. This separation is inspired by cultural evolution, where individual innovation and the social spread of useful innovations through a population jointly drive adaptation [42, 51].

The balance between individual and social learning, the strategy to select a node (random or best fit), the size of the neighborhood, and the type of neighborhood (closest neighbors, distance-proportional sampling, or random selection) are key design choices that shape how information flows through the graph. A small neighborhood with strong social learning concentrates exploitation among very similar tasks, while a larger neighborhood or weaker social learning permits broader exploration. Understanding which of these choices matters and whether a single robust configuration exists in different problem domains is essential for the practical application of MONET.

To this end, we conducted a hyperparameter study in four environments: archery, arm, and cartpole with 5,000 tasks each, and hexapod with 2,000 tasks. We show that MONET outperforms both MT-ME and PT-ME across all four environments, demonstrating that exploiting task-space structure through a network of task neighborhoods is an effective approach to multi-task optimization at scale. Our contributions are:

1. A unified formal taxonomy of MAP-Elites variants for multi-task optimization that exposes the structural modifications distinguishing each variant and motivates MONET (Section 2).
2. MONET, a graph-based multi-task optimizer that makes task-space neighborhoods an explicit element of recombination and combines localized social and individual learning (Section 3).
3. A systematic hyperparameter study across four task domains (archery, arm, cartpole, and hexapod) analyzing the effects of neighborhood type, neighborhood size, and the social/individual balance (Sections 4 and 5).
4. Empirical evidence that a single MONET configuration generalizes across all four domains and outperforms MT-ME and PT-ME, showing task-space topology to be a valuable and underexploited source of information in multi-task optimization (Sections 5 and 6).

2 Related Works

Evolutionary multi-task optimization has become a highly active area of research within the computational intelligence community [23, 24, 40, 53]. To concretize the description, we specify the problem setting as follows. Consider a set of n tasks, each described by a parameter vector $\boldsymbol{\tau}_i \in \mathcal{T}$, for which we want to find the corresponding solutions $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_n \in \Theta$ simultaneously. The goal of multi-task

optimization is to find, for every task $\boldsymbol{\tau} \in \mathcal{T}$, an optimal solution as:

$$\boldsymbol{\theta}^*_{\boldsymbol{\tau}} = \arg\max_{\boldsymbol{\theta} \in \Theta} f(\boldsymbol{\theta}, \boldsymbol{\tau}), \tag{1}$$

where Θ is the solution space, $\mathcal{T}$ the task parameter space and $f : \Theta \times \mathcal{T} \to \mathbb{R}$ the fitness function.

Existing methodologies generally fall into two categories: *implicit transfer*, which utilizes a single, unified search space for all tasks, and *explicit transfer*, which operates across distinct solution spaces. Although implicit transfer is relatively straightforward to implement, its application is largely restricted to homogeneous environments. In contrast, explicit transfer accommodates heterogeneous tasks, though determining the optimal knowledge to exchange between distinct spaces remains a complex challenge. To facilitate this transfer, contemporary frameworks have leveraged probabilistic search distributions, search direction vectors, higher-order heuristics, and surrogate models. A representative example is the Multi-Factorial Evolutionary Algorithm II (MFEA-II) [7], which learns an online inter-task transfer parameter to modulate the probabilistic crossover between tasks and thereby suppress negative transfer during multifactorial search.

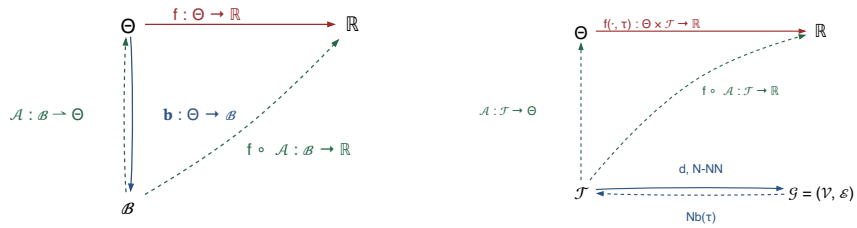
Historically, the application of these methods has been limited to optimizing a small number of tasks concurrently (typically between two and ten). As catalogued in the evolutionary multi-tasking (EMT) survey of [40], examples include optimizing three robot morphologies via *neuroevolution* [7], two symbolic regression datasets using genetic programming [54], ten numerical functions for software testing [43], multi-UAV path planning across three tasks [6], power dispatch optimization for two bus systems [31], logistics planning for four package delivery problems [22], and the design of three diode models [30]. Similar multi-task formulations exist in other fields, such as Bayesian optimization [41], which is frequently used to tune machine learning models across multiple datasets. However, because Bayesian approaches rely on Gaussian processes that scale cubically[3], they are restricted to computationally expensive fitness functions and struggle to scale beyond a thousand evaluations [45]. To address these scalability limits, recent works in evolutionary many-task optimization have successfully tackled increasingly larger problem sets, scaling to 30, 50, 500, and up to 2,000 tasks simultaneously. Aligning with this trajectory toward large-scale optimization.

In order to show the connection of the proposed method to the existing literature, we first introduce a unified notation framework for the whole MAP-Elites algorithm family, enabling each multi-task variant to be presented as a modification of its underlying formulation[4]. The same base is then extended once more in Section 3 when MONET is defined.

Definition 1 (MAP-Elites). *Let Θ denote a solution space, $f : \Theta \to \mathbb{R}$ a fitness function to be* maximized*, $\mathcal{B}$ a behavior space, and $\mathbf{b} : \Theta \to \mathcal{B}$ a behavioral*

[3] In addition, the approaches such as Nyström approximation have limitations.

[4] We adopt the maximization convention throughout; a minimization problem with cost c is handled by setting $f = -c$.



(a) MAP-Elites (Definition 1). Solid blue: behavioral descriptor $\mathbf{b} : \Theta \to \mathcal{B}$; solid red: fitness $f : \Theta \to \mathbb{R}$; dashed green: archive $\mathcal{A} : \mathcal{B} \rightharpoonup \Theta$ and composed map $f \circ \mathcal{A} : \mathcal{B} \to \mathbb{R}$.

(b) MONET (Definition 2). The behavioral descriptor $\mathbf{b}$ is absent; solid red: fitness $f(\cdot, \boldsymbol{\tau}) : \Theta \times \mathcal{T} \to \mathbb{R}$; dashed green: archive $\mathcal{A} : \mathcal{T} \to \Theta$ and $f \circ \mathcal{A} : \mathcal{T} \to \mathbb{R}$; solid blue: graph construction via task distance d; dashed blue: neighborhood lookup $\mathrm{Nb}(\boldsymbol{\tau})$.

Fig. 2: Commutative diagrams comparing MAP-Elites and MONET.

descriptor that maps a solution to its behavior. Fix a finite discretization of $\mathcal{B}$ into cells. MAP-Elites [14, 38] maintains an archive $\mathcal{A}$[5], treated as a partial function from cells to Θ, initialized empty and updated by the following elitist rule: given a candidate $\boldsymbol{\theta} \in \Theta$, insert $\boldsymbol{\theta}$ into the cell containing $\mathbf{b}(\boldsymbol{\theta})$ iff that cell is empty or $f(\boldsymbol{\theta}) > f(\mathcal{A}[\mathbf{b}(\boldsymbol{\theta})])$. The run returns $\mathcal{A}$, which trades objective value against behavioral coverage rather than optimizing f alone.

Figure 2a summarizes the mappings of Definition 1 as a commutative diagram.[6] Now we are ready to introduce relevant multi-task variants as extensions of this definition: each paragraph names exactly what is changed and leaves the rest of the definition in place.

Multi-Task MAP-Elites (MT-ME) [39]. MT-ME extends Definition 1 by *replacing* the behavioral descriptor $\mathbf{b} : \Theta \to \mathcal{B}$ with the task parameter itself. A finite set of tasks $\{\boldsymbol{\tau}_1, \ldots, \boldsymbol{\tau}_n\} \subset \mathcal{T}$ is fixed in advance, the archive becomes $\mathcal{A} : \mathcal{T} \to \Theta$ indexed directly by task, and the fitness lifts to $f : \Theta \times \mathcal{T} \to \mathbb{R}$, matching the problem statement of Eq. (1). Recombination draws parents globally from the entire task archive, so information is shared across all tasks regardless of their similarity. MT-ME uses the iso-line-dd variation operator [47], which exploits the observation that elites tend to cluster in a compact "elite

[5] We use uppercase calligraphic $\mathcal{A}$ because the archive is primarily a data structure that is *treated as* a partial function; this convention is standard in the MAP-Elites literature and consistent with the use of uppercase for structured objects such as operators or transforms.

[6] Note that Definition 1 uses a single-task fitness $f : \Theta \to \mathbb{R}$; the multi-task extensions below lift this to $f : \Theta \times \mathcal{T} \to \mathbb{R}$ by conditioning on a task parameter $\boldsymbol{\tau} \in \mathcal{T}$, recovering the problem statement of Eq. (1).

hypervolume" in genotype space: given two parent elites $\boldsymbol{\theta}_i$ and $\boldsymbol{\theta}_j$, a candidate is generated by adding isotropic Gaussian noise and a directional component along the line connecting the parents, thereby biasing search toward the region of genotype space where elites concentrate.

Multi-Task Multi-Behavior MAP-Elites (MTMB-ME) [3]. MTMB-ME extends MT-ME by *restoring* the behavioral descriptor from Definition 1 on top of the task index, yielding a two-level archive $\mathcal{A} : \mathcal{T} \times \mathcal{B} \to \Theta$ indexed jointly by task and behavior. Task-specific adaptation and behavioral diversity are therefore maintained simultaneously within a single archive. Anne and Mouret [3] applied MTMB-ME to a humanoid robot fault-recovery scenario, identifying successful contact positions on a wall that help the robot regain balance after a leg fault, and outperformed random search, grid search, and standard MAP-Elites both in the number of tasks solved and in the number of solutions found per task (about twice as many).

Parametric-Task MAP-Elites (PT-ME) [4]. PT-ME extends MT-ME to *continuous* task spaces. Instead of fixing a finite set of tasks in advance, PT-ME draws a fresh task $\boldsymbol{\tau} \sim \mathcal{T}$ at every iteration and stores, for each drawn task, the best solution encountered so far, so the archive becomes a dense dataset covering the entire task space. PT-ME alternates between two variation operators at equal probability. The first is SBX [16] with a bandit-tuned tournament that biases task selection toward tasks close to the parents, replacing the iso-line-dd operator of MT-ME. The second, and more distinctive, operator is a *local linear regression*: PT-ME partitions the task space into cells via Centroidal Voronoi Tessellation (CVT) [19, 48], precomputes a k-dimensional tree (k-d tree) [10] for fast cell lookup and a Delaunay triangulation [17] for adjacency, then fits a linear least-squares model $M = (\mathbf{T}^\top \mathbf{T})^{-1} \mathbf{T}^\top \mathbf{X}$ to the elites in adjacent cells, where $\mathbf{T}$ stacks the task vectors of the adjacent cells and $\mathbf{X}$ the corresponding solution vectors; a candidate is then generated as $\boldsymbol{\theta} = M \cdot \boldsymbol{\tau} + \sigma_{\text{reg}} \cdot \mathcal{N}(0, \text{var}(\mathbf{X}))$. This regression operator is notable because it explicitly exploits task-space locality: the Delaunay triangulation defines a geometric neighborhood over $\mathcal{T}$, and the linear model builds a local approximation of the task-to-solution mapping from that neighborhood. On parametric toy problems and a robotic simulation, PT-ME yields improved performance over standard MAP-Elites and Proximal Policy Optimization (PPO) [2].

MAP-Elites with Knowledge Transfer (MMKT) [32]. This variant extends MT-ME by *partitioning* the task set into groups $\{\mathcal{T}_1, \ldots, \mathcal{T}_m\}$ obtained by applying k-means clustering to the task parameters, and then promoting knowledge transfer between tasks through linear combinations of solutions drawn from the same group. A preliminary study on the robotic arm task showed this group-based transfer to outperform ungrouped MT-ME under the same evaluation budget.

Adaptive Group-based Collaborative Optimization (AGCO) [27]. This framework extends the group-based view further with a two-stage transfer protocol. An *inter-group knowledge separation* stage employs an adaptive selection mechanism for crossover operators to control information exchange between

groups, and an *intra-group knowledge reunion* stage leverages Thompson sampling to weight within-group recombination so that target tasks assimilate insights from multiple in-group sources according to their evolutionary preferences.

Most of the variants above treat the archive as fully connected: every elite can serve as a parent for any cell regardless of the relationship between the corresponding tasks. PT-ME is a partial exception: its regression operator builds a local linear model from Delaunay-adjacent cells, thereby exploiting geometric locality in $\mathcal{T}$ for candidate generation. However, this locality is confined to one of two variation operators (applied with 50% probability), the adjacency structure is a by-product of the CVT tessellation rather than an explicit design element, and the SBX path still draws parents globally. An alternative that makes locality a first-class principle is familiar from *cellular evolutionary algorithms* (cEAs) [1], which organize the population into local neighborhoods and restrict recombination accordingly, so that information propagates through the population only via those neighborhoods. MONET (Section 3) follows the same intuition but imposes the graph over tasks rather than over individuals: it extends MT-ME by replacing the implicit fully-connected archive $\mathcal{A} : \mathcal{T} \to \Theta$ with an explicit graph structure over $\mathcal{T}$, inducing neighborhoods from task-space proximity and mediating *all* recombination through them.

3 MONET

Building on the taxonomy introduced in Section 2, we now introduce MONET. Figure 1 illustrates the task-graph representation at the core of MONET: nodes are tasks, edges connect neighboring tasks in the task parameter space, and each node stores the best solution found so far.

Definition 2 (MONET). *Let $\{\boldsymbol{\tau}_1, \ldots, \boldsymbol{\tau}_n\} \subset \mathcal{T}$ be a finite task set, $f : \Theta \times \mathcal{T} \to \mathbb{R}$ a task-parameterized fitness function, and $d : \mathcal{T} \times \mathcal{T} \to \mathbb{R}_{\geq 0}$ a task distance. MONET represents the optimization state as a labeled graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A})$ whose nodes $\mathcal{V} = \{\boldsymbol{\tau}_1, \ldots, \boldsymbol{\tau}_n\}$ are the tasks themselves, whose edges $\mathcal{E} = \{(\boldsymbol{\tau}_i, \boldsymbol{\tau}_j) : \boldsymbol{\tau}_j$ is among the N nearest neighbors of $\boldsymbol{\tau}_i$ under $d\}$ encode task-space proximity for a user-chosen neighborhood size N, and whose labeling $\mathcal{A} : \mathcal{V} \to \Theta$ stores the best solution found so far for each task. Candidate solutions are admitted by the elitist rule: a candidate $\boldsymbol{\theta}$ generated for task $\boldsymbol{\tau}_i$ replaces $\mathcal{A}[\boldsymbol{\tau}_i]$ iff $f(\boldsymbol{\theta}, \boldsymbol{\tau}_i) \geq f(\mathcal{A}[\boldsymbol{\tau}_i], \boldsymbol{\tau}_i)$.*

Figure 2b summarizes the mappings of Definition 2. Compared with the base MAP-Elites diagram (Figure 2a), the behavioral descriptor $\mathbf{b}$ is absent and the archive is indexed directly by task; the fitness is conditioned on a task parameter; and the bottom row shows the graph construction from the task distance d and the neighborhood lookup $\mathrm{Nb}(\boldsymbol{\tau})$ that mediates social learning.

Candidates are generated by one of two complementary mechanisms, selected at each step with probability p_{ind}. *Individual learning* mutates the current label $\mathcal{A}[\boldsymbol{\tau}_i]$ of a focal task, exploring the local fitness landscape of $\boldsymbol{\tau}_i$ in isolation. *Social learning* draws a neighbor $\boldsymbol{\tau}_j$ from the graph neighborhood of $\boldsymbol{\tau}_i$ and recombines

Algorithm 1 MONET

Require: Task set $T = \{\boldsymbol{\tau}_1, \ldots, \boldsymbol{\tau}_n\}$, fitness function f, task distance d
Require: Budget E, init fraction K_{init}, neighborhood size N, selection strategy SELECT$(\cdot)$, probability p_{ind}
Ensure: Archive $\mathcal{A}$ mapping each task to its best solution

```
A[τ] ← ∅ for all τ ∈ T                                  ▷ empty network
for each τ_i ∈ T do                                     ▷ build k-NN task graph
    Nb(τ_i) ← N nearest tasks under d
end for
for i ← 1 to K_init do                                  ▷ random initialization
    τ ← RandomTask(T)
    θ ~ U(p_min, p_max)
    f_x ← f(θ, τ)
    if A[τ] = ∅ or f_x ≥ A[τ].fitness then              ▷ elitist update
        A[τ] ← (θ, f_x)
    end if
end for
e ← K_init                                              ▷ main loop
while e < E do
    τ ← RandomTask(T)                                   ▷ sample focal task
    if rand() < p_ind then                              ▷ individual learning
        Mutation(τ, A)                                  ▷ IndividualLearning(τ, A)
        e ← e + 1
    else                                                ▷ social learning
        τ_j ← Select(Nb(τ))                             ▷ pick one neighbor
        if τ_j ≠ nil then                               ▷ skip if no graph neighbor exists
            SBX(τ, τ_j, A)                              ▷ SocialLearning(τ, τ_j, A)
            e ← e + 1
        end if
    end if
end while
return A
```

$\mathcal{A}[\boldsymbol{\tau}_i]$ with $\mathcal{A}[\boldsymbol{\tau}_j]$, transferring information between similar tasks along the edges of $\mathcal{G}$. Empty nodes are initialized by individual learning with a uniform random solution, or by social learning by copying from a non-empty neighbor (with a uniform random fallback if no such neighbor exists); social learning is skipped at a step where the chosen neighbor is itself empty.

In this work, we use the Euclidean distance in $\mathcal{T}$ as the task distance d, and set the neighborhood size N (the number of nearest neighbors retained per task, Def. 2) to a small fraction of $|\mathcal{V}|$ (see Section 4), so that $\mathcal{G}$ is a sparse, partially connected graph whose density is controlled by the chosen neighborhood size. Algorithm 1 provides the pseudocode of MONET and the code is publicly accessible [25].

4 Experimental Setup

We compare MONET against PT-ME and MT-ME across four task environments: archery, arm, cartpole, and hexapod (Figure 3).

PT-ME samples tasks continuously from $\mathcal{T}$ rather than operating over a fixed task set: it partitions $\mathcal{T}$ into n cells via Centroidal Voronoi Tessellation (CVT) [19, 48] and, by the end of its run, each cell holds the highest-fitness task-solution pair it has sampled. To place the three algorithms on the same footing, we take these n cell elites ($n = 5000$ for archery, arm, and cartpole, and $n = 2000$ for hexapod) as the fixed task set on which MT-ME and MONET are then run. This removes task selection as a confound in the cross-algorithm comparison. One asymmetry remains: under this protocol each PT-ME task is evaluated essentially once (the sample whose solution became its cell's elite), whereas MT-ME and MONET revisit each task many times to refine its solution, so the total evaluation budget is identical across algorithms but the per-task refinement opportunity is not.

For PT-ME and MT-ME we use the default parameters from [4, 39]. For MONET, we first conduct a hyperparameter sensitivity analysis. We consider the following hyperparameters and ranges: $p_{\text{ind}} \in \{0, 0.3, 0.5, 0.7, 1\}$, strategy $\in$ {random, best_fitness}, neighborhood $\in$ {random, closest, distance_proportional}, and neighbor_percentage $\in \{1\%, 5\%, 10\%\}$. Here p_{ind} is the per-step probability of performing individual learning (mutation of the focal task's elite) rather than social learning (crossover with a neighbor's elite); strategy is the rule used to pick the neighbor during social learning, either uniformly at random or deterministically the neighbor with the highest current fitness; neighborhood determines how the neighbor set of each task is built, taking the N closest tasks in $\mathcal{T}$, N uniformly random tasks, or N tasks drawn without replacement with probability proportional to task-space similarity; and neighbor_percentage sets this neighborhood size as a fraction $N = \lceil \text{neighbor_percentage} \cdot |\mathcal{V}| \rceil$ of the total task set.

For each hyperparameter combination we perform up to 20 independent runs with different random seeds and report the mean fitness with standard deviation as well as the area under the learning curve (AUC), capturing both final performance and cumulative learning efficiency. The sensitivity sweep is concentrated on archery and arm (20 seeds per configuration); cartpole and hexapod are evaluated on a smaller subset of configurations because of simulator cost.

To quantify the importance of individual hyperparameters and their interactions, both per task and aggregated across all tasks, we fit a gradient-boosted tree surrogate model and compute SHapley Additive exPlanations (SHAP) values [33, 34], following the surrogate-based hyperparameter importance methodology of [28]. Guided by this analysis, we then assign reasonable values to each hyperparameter, yielding a single configuration of MONET that we refer to as the *default* configuration and compare it against PT-ME and MT-ME. Additionally, we employ the Sequential Parameter Optimization Toolbox (SPOT) [9], a surrogate-based optimizer, to automatically tune MONET's hyperparameters per task and include these task-specific configurations in the comparison.

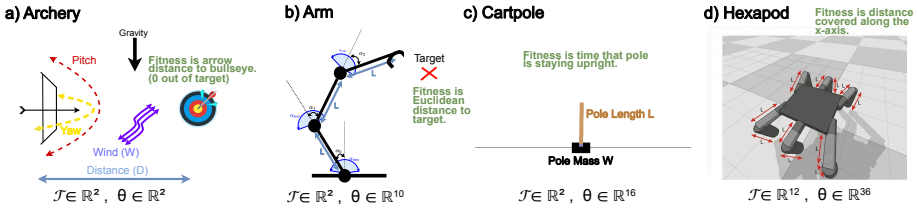


Fig. 3: Overview of benchmark environments.

To assess statistical significance of the performance differences between algorithms, we apply the Mann-Whitney U test [35] to the final fitness values and AUC values across all seeds and report the resulting p-values with Holm-Bonferroni correction [20, 26] for multiple comparisons.

Higher fitness is better in all four benchmarks, but the natural scale differs across them: archery and arm return values in $[0, 1]$; cartpole reports the mean balanced-step count across 10 rollouts and therefore lies in $[0, t_{\max}]$ with $t_{\max} = 1000$; hexapod reports the forward centre-of-mass displacement in metres over a 3 s episode with no formal upper bound (in practice below 1.5 m). Closed-form fitness expressions are defined in Appendix C. All four benchmarks are run for 10^6 evaluations with a batch size of 64, replicated over 20 random seeds per (algorithm, domain) pair. Additional supplementary material is available online [25].

Archery. A projectile-aiming task in which the archer compensates for target distance and wind (Figure 3b) [4]. Tasks (2D) vary the distance $D \in [5, 40]$ m and horizontal wind $W \in [-10, 10]$ m/s^2; solutions (2D) are yaw and pitch in $[-\pi/12, \pi/12]$ at fixed launch speed $v_0 = 70$ m/s.

Arm. A planar 10-degree-of-freedom (10-DoF) arm places its end-effector at a fixed target (Figure 3a) [4, 39]. Tasks (2D) vary the link length L and maximum joint angle $\alpha_{\max}$, both in $[0, 1]$; solutions (10D) are the 10 joint angles.

Cartpole. Cart-pole balancing under varying physical parameters (Figure 3c) [8, 36]. Tasks (2D) vary pole mass $m_p \in [0.05, 0.5]$ kg and pole length $L \in [0.3, 1.2]$ m; solutions (16D) are a direct encoding of the weights of a small multi-layer perceptron controller (4 inputs, 2 hidden tanh units, 2 output logits, with bias units on the input and hidden layers).

Hexapod. A simulated hexapod walks forward across 2 000 random morphologies (Figure 3d) [14, 39], simulated in PyBullet [13]. Tasks (12D) are 12 leg-segment length changes; solutions (36D) parameterise a gait controller.

5 Results

Across the four domains, MONET matches or exceeds the strongest MAP-Elites baseline on final mean fitness, with the largest gap on the hexapod task where the per-domain tuned configuration nearly doubles the final fitness of MT-ME (Figure 4, Table 1). We first walk through the per-domain comparison, then unpack the hyperparameter analysis that produced the shared default configuration.

5.1 Algorithm Comparison

We report two MONET variants: $MONET_{\text{default}}$, a single configuration shared across domains (defined in Section 5.2), and $MONET_{\text{tuned}}$, a per-domain tuning produced with SPOT [9]. Both are compared against PT-ME and MT-ME with their published defaults. Each box in Figure 4 aggregates 20 seeds per (algorithm, domain) pair, and brackets mark Mann-Whitney U with Holm-Bonferroni correction.

Archery. MT-ME, PT-ME and $\text{MONET}_{\text{tuned}}$ all saturate at the maximum fitness (MT-ME and PT-ME at 1.000; $\text{MONET}_{\text{tuned}}$ at 0.9999 ± 0.0001); $\text{MONET}_{\text{default}}$ stops slightly short at 0.996 ± 0.000. Because residual variance collapses at saturation, even 10^{-4}-level gaps between saturating runs register as highly significant under Mann-Whitney U despite negligible practical effect, so we rely on AUC as the discriminating signal. $\text{MONET}_{\text{default}}$ reaches an AUC of 976.3 ± 0.6, significantly above MT-ME (957.2 ± 3.3, $p < 0.001$) and PT-ME (924.1 ± 1.1, $p < 0.001$). $\text{MONET}_{\text{tuned}}$ improves AUC marginally to 979.2 ± 1.9. MONET therefore reaches near-optimal performance substantially earlier than the baselines.

Arm. $\text{MONET}_{\text{default}}$ reaches 0.770 ± 0.001 on final fitness, just below MT-ME (0.772 ± 0.002, $p < 0.001$) and well above PT-ME (0.747 ± 0.001, $p < 0.001$). On AUC it is the strongest of the four algorithms: 762.7 ± 1.2, ahead of MT-ME (760.1 ± 3.4, $p = 0.003$) and PT-ME (734.9 ± 2.6, $p < 0.001$). This reflects faster convergence in the early phase (Figure 4b). $\text{MONET}_{\text{tuned}}$ edges final fitness marginally higher to 0.773 ± 0.000.

Cartpole. $\text{MONET}_{\text{default}}$ reaches 997.1 ± 2.5, statistically indistinguishable from MT-ME (994.5 ± 8.5, $p = 0.69$) and PT-ME (988.0 ± 9.1, $p = 0.22$). MT-ME retains a significant AUC lead (978.4 ± 11.4 vs. 969.0 ± 7.1 for $\text{MONET}_{\text{default}}$, $p < 0.001$): it converges earlier, while MONET terminates higher. $\text{MONET}_{\text{tuned}}$ effectively solves the task (1000.0 ± 0.0, $p < 0.05$ vs. both baselines).

Hexapod. $\text{MONET}_{\text{default}}$ (0.695 ± 0.111) performs significantly better than MT-ME (0.530 ± 0.032, $p < 0.001$) and statistically indistinguishable from PT-ME (0.727 ± 0.078, $p = 0.35$). Tuning the parameters of MONET improves performance further to 0.956 ± 0.082, significantly above both baselines ($p < 0.001$ vs. MT-ME; $p < 0.001$ vs. PT-ME).

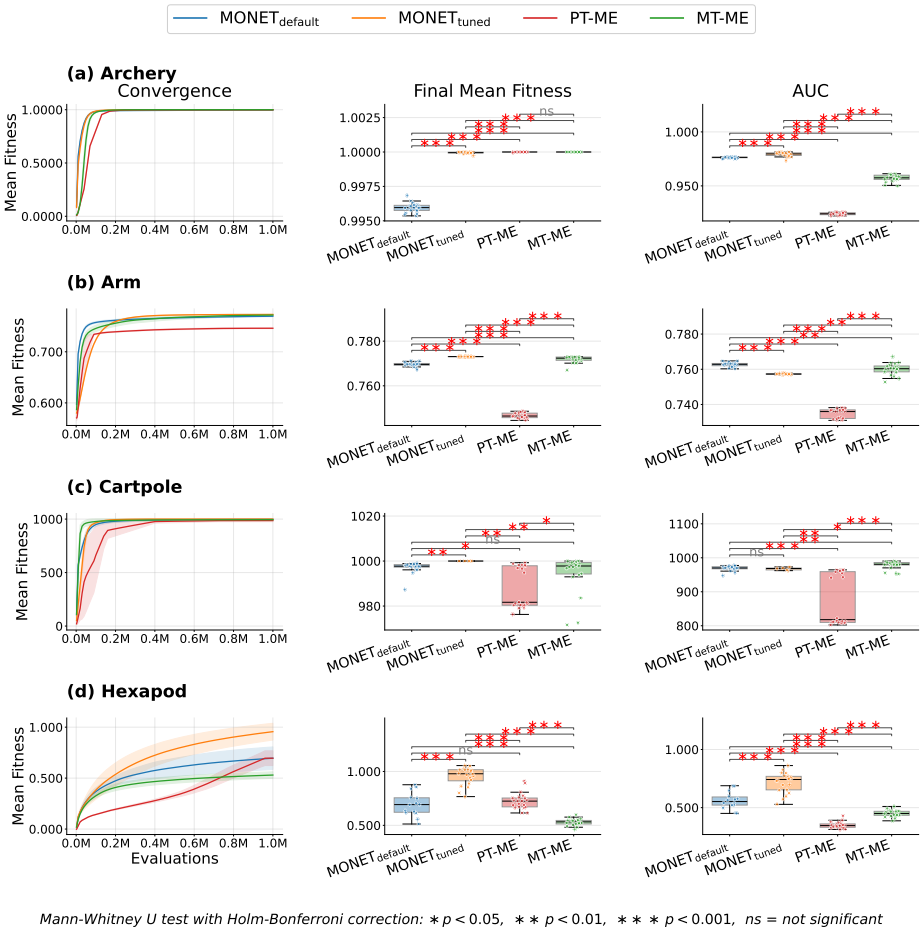


Fig. 4: Algorithm comparison across the four benchmark domains.

Previous work reported that PT-ME outperformed MT-ME [4], whereas in our experiments, MT-ME achieved better performance than PT-ME. This discrepancy arises because PT-ME samples tasks continuously from the task space, which can result in selecting tasks that are easier to solve. To ensure a fair comparison, we fixed the task set to the final tasks discovered by PT-ME and evaluated both methods on this same set of tasks.

5.2 Hyperparameter Sensitivity and Default Configuration

The most important finding from the SHAP-based sensitivity analysis is that the probability of individual learning p_{ind} accounts for most of the variation in performance, with a mean absolute SHAP value $\overline{|\text{SHAP}|} = 0.69$, followed by strategy (0.14), neighbor_percentage (0.13), and neighborhood (0.09); see Figure 5. Low

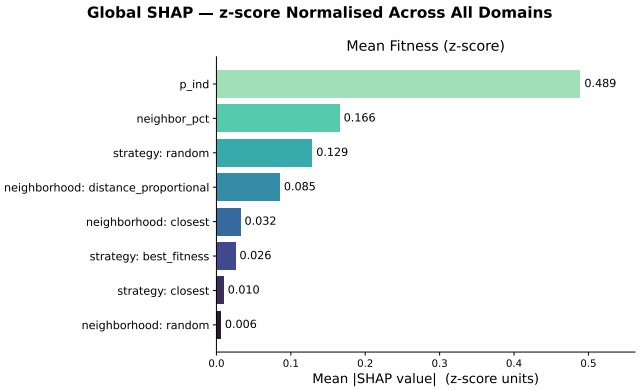


Fig. 5: Global hyperparameter importance for MONET: mean absolute SHAP value on a gradient-boosted tree surrogate, with final mean fitness z-scored per domain and averaged across all four. p_{ind} dominates by an order of magnitude.

values of p_{ind} (favoring social learning over individual mutation) consistently improve both mean fitness and AUC across all domains, supporting the central design intuition behind MONET that information transfer between similar tasks is the dominant lever. The remaining hyperparameters show weaker and more domain-dependent effects (see SI for detailed SHAP analysis per domain [25]).

The default parameters of MONET were defined based on the SHAP analysis as: strategy = best_fitness, neighborhood = closest, $p_{\text{ind}} = 0.3$, and neighbor_percentage = 5% (obtained by ranking configurations on the z-scored mean fitness across all domains, see SI [25]). This configuration picks the fittest neighbor for crossover, draws neighbors from the k nearest tasks in task space, and mixes mostly social learning (70%) with occasional individual refinement (30%). The SPOT-tuned per-domain configurations differ markedly from the default and from each other: archery uses a large neighborhood (39%) with moderate $p_{\text{ind}} = 0.22$; arm uses a very small neighborhood (0.1%) with high $p_{\text{ind}} = 0.54$ (favouring individual refinement on this well-conditioned task); cartpole uses a small neighborhood (1%) with a balanced $p_{\text{ind}} = 0.5$; and hexapod uses a small neighborhood (1%) with pure social learning ($p_{\text{ind}} = 0$). This spread confirms that the optimal individual vs. social learning balance is task-dependent. The default configuration is competitive with the baselines: it achieves the best AUC on three of four domains (archery, arm, hexapod), though it falls slightly short on archery final fitness and on cartpole AUC, where the saturating baselines retain an edge (see Fig. 4 and Table 1).

6 Conclusion

In this work, we propose MONET for finding high-quality solutions for a large set of related tasks simultaneously. We structure the task space as a network of

Table 1: Final mean fitness and AUC ($\times 10^3$ on the $[0, 1]$ domains) per domain. Mean $\pm$ std over 20 random seeds per (algorithm, domain) pair. Best per domain in **bold**.

	Archery	Arm	Cartpole	Hexapod
(a) Final mean fitness				
PT-ME	1.000 ± 0.000	0.747 ± 0.001	988.0 ± 9.1	0.727 ± 0.078
MT-ME	1.000 ± 0.000	0.772 ± 0.002	994.5 ± 8.5	0.530 ± 0.032
$\text{MONET}_{\text{default}}$	0.996 ± 0.000	0.770 ± 0.001	997.1 ± 2.5	0.695 ± 0.111
$\text{MONET}_{\text{tuned}}$	1.000 ± 0.000	$\mathbf{0.773 \pm 0.000}$	$\mathbf{1000.0 \pm 0.0}$	$\mathbf{0.956 \pm 0.082}$
(b) AUC ($\times 10^3$ on $[0, 1]$ domains)				
PT-ME	924.1 ± 1.1	734.9 ± 2.6	876.6 ± 74.8	349.9 ± 27.7
MT-ME	957.2 ± 3.3	760.1 ± 3.4	$\mathbf{978.4 \pm 11.4}$	450.2 ± 29.3
$\text{MONET}_{\text{default}}$	976.3 ± 0.6	$\mathbf{762.7 \pm 1.2}$	969.0 ± 7.1	560.9 ± 74.5
$\text{MONET}_{\text{tuned}}$	$\mathbf{979.2 \pm 1.9}$	757.3 ± 0.2	967.6 ± 4.4	$\mathbf{716.0 \pm 87.8}$

neighborhoods that exploits task similarity for knowledge transfer, and introduce two operators: individual learning refines a task's current elite in isolation, while social learning exchanges knowledge across neighboring tasks. Across four domains, this yields an advantage over flat-archive baselines that ignore task-space topology.

The sensitivity analysis showed that the probability of individual learning p_{ind} dominates performance, with high values hurting it, implying that social learning drives most of the knowledge transfer. Our sweep covered only a fixed grid of p_{ind} values, and per-domain tuning with SPOT then selected $p_{\text{ind}} = 0.22$ for archery, 0.54 for arm, 0.5 for cartpole, and 0 for hexapod, indicating that small amounts of individual refinement remain useful when per-task landscapes are well-conditioned.

We established task connectivity from task-parameter similarity, which assumes that parameter-similar tasks benefit from shared solutions. This need not always hold: in hexapod, for example, small changes to front-right and back-left leg lengths yield nearby task vectors but can demand qualitatively different controllers. Future work should therefore explore alternative connectivity definitions and assess their effect on knowledge transfer.

MONET currently has four hyperparameters, and a natural next step is online self-adaptive parameter control (e.g., adapting the neighborhood size as in MT-ME's task-competition and replacement rule [39], or dynamically switching between individual and social learning strategies [50]) to generalize better across domains. The space of individual and social learning operators can also be enriched with strategies that converge to more generalist solutions performing well across wide task ranges [46].

References

1. Alba, E., Dorronsoro, B.: Cellular Genetic Algorithms, Operations Research/Computer Science Interfaces Series, vol. 42. Springer US, Boston, MA (2008). https://doi.org/10.1007/978-0-387-77610-1
2. and Filip Wolski and Prafulla Dhariwal and Alec Radford and Oleg Klimov, J.S.: Proximal policy optimization algorithms. CoRR **abs/1707.06347** (2017)
3. Anne, T., Mouret, J.B.: Multi-task multi-behavior MAP-elites. In: Proceedings of the Companion Conference on Genetic and Evolutionary Computation. pp. 111–114. GECCO '23 Companion, Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3583133.3590730
4. Anne, T., Mouret, J.B.: Parametric-Task MAP-Elites. In: Proceedings of the Genetic and Evolutionary Computation Conference. pp. 68–77 (Jul 2024). https://doi.org/10.1145/3638529.3653993
5. Anne, T., Mouret, J.B.: Parametric-Task MAP-Elites. https://github.com/hucebot/Parametric-Task_MAP-Elites (2024)
6. Bali, K.K., Gupta, A., Ong, Y.S., Tan, P.S.: Cognizant multitasking in multiobjective multifactorial evolution: MO-MFEA-II. IEEE Transactions on Cybernetics **51**(4), 1784–1796 (2021). https://doi.org/10.1109/TCYB.2020.2981733
7. Bali, K.K., Ong, Y.S., Gupta, A., Tan, P.S.: Multifactorial evolutionary algorithm with online transfer parameter estimation: MFEA-II. IEEE Transactions on Evolutionary Computation **24**(1), 69–83 (2020). https://doi.org/10.1109/TEVC.2019.2906927
8. Barto, A.G., Sutton, R.S., Anderson, C.W.: Neuronlike adaptive elements that can solve difficult learning control problems. IEEE Transactions on Systems, Man, and Cybernetics **SMC-13**(5), 834–846 (1983). https://doi.org/10.1109/TSMC.1983.6313077
9. Bartz-Beielstein, T.: Optimization with spotoptim (2026), https://arxiv.org/abs/2604.13672
10. Bentley, J.L.: Multidimensional binary search trees used for associative searching. Communications of the ACM **18**(9), 509–517 (1975). https://doi.org/10.1145/361002.361007
11. Bongard, J., Zykov, V., Lipson, H.: Resilient machines through continuous self-modeling. Science **314**(5802), 1118–1121 (2006). https://doi.org/10.1126/science.1133687
12. Chatzilygeroudis, K., Cully, A., Vassiliades, V., Mouret, J.B.: Quality-diversity optimization: A novel branch of stochastic optimization. In: Black Box Optimization, Machine Learning, and No-Free Lunch Theorems, pp. 109–135. Springer Optimization and Its Applications, Springer (2021). https://doi.org/10.1007/978-3-030-66515-9_4
13. Coumans, E., Bai, Y.: PyBullet, a Python module for physics simulation for games, robotics and machine learning. http://pybullet.org (2021)
14. Cully, A., Clune, J., Tarapore, D., Mouret, J.B.: Robots that can adapt like animals. Nature **521**(7553), 503–507 (2015). https://doi.org/10.1038/nature14422, https://doi.org/10.1038/nature14422
15. Deb, K.: Multi-Objective Optimization Using Evolutionary Algorithms. Wiley, Chichester, UK (2001)
16. Deb, K., Agrawal, R.B.: Simulated binary crossover for continuous search space. Complex Systems **9**(2), 115–148 (1995)

17. Delaunay, B.: Sur la sphere vide. In: Proceedings of the Mathematics, Toronto. pp. 695–700. Toronto (1928)
18. van Diggelen, F., Cambier, N., Ferrante, E., Eiben, A.E.: A model-free method to learn multiple skills in parallel on modular robots. Nature Communications **15**, 6267 (2024). https://doi.org/10.1038/s41467-024-50131-4
19. Du, Q., Faber, V., Gunzburger, M.: Centroidal voronoi tessellations: Applications and algorithms. SIAM Review **41**(4), 637–676 (1999). https://doi.org/10.1137/S0036144599352836
20. Dunn, O.J.: Multiple comparisons among means. Journal of the American Statistical Association **56**(293), 52–64 (1961). https://doi.org/10.1080/01621459.1961.10482090
21. Feller, W.: An Introduction to Probability Theory and Its Applications, vol. 1. Wiley, 3 edn. (1968)
22. Feng, L., Huang, Y., Zhou, L., Zhong, J., Gupta, A., Tang, K., Tan, K.C.: Explicit evolutionary multitasking for combinatorial optimization: A case study on capacitated vehicle routing problem. IEEE Transactions on Cybernetics **51**(6), 3143–3156 (2021). https://doi.org/10.1109/TCYB.2019.2962865
23. Gupta, A., Ong, Y.S., Feng, L.: Multifactorial evolution: Toward evolutionary multitasking. IEEE Transactions on Evolutionary Computation **20**(3), 343–357 (2016). https://doi.org/10.1109/TEVC.2015.2458037
24. Gupta, A., Zhou, L., Ong, Y.S., Chen, Z., Hou, Y.: Half a dozen real-world applications of evolutionary multitasking, and more. IEEE Computational Intelligence Magazine **17**, 49–66 (May 2022). https://doi.org/10.1109/MCI.2022.3155332
25. Hatzky, J., Bartz-Beielstein, T., Eiben, A.E., Yaman, A.: Repository for MONET: Multi-Task Optimization over Networks of Tasks. https://github.com/ju2ez/MONET/tree/PPSN-MONET-introduction (2026)
26. Holm, S.: A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics **6**(2), 65–70 (1979)
27. Hou, Y., Yu, Z., Guo, Z., Pei, W., Wu, Y., Ge, H., Xue, B., Zhang, M.: A Group-Based Many-Task Collaborative Optimization Framework for Evolutionary Robots Design. IEEE Transactions on Systems, Man, and Cybernetics: Systems **55**(5), 3492–3505 (May 2025). https://doi.org/10.1109/TSMC.2025.3541002
28. Hutter, F., Hoos, H., Leyton-Brown, K.: An efficient approach for assessing hyperparameter importance. In: Proceedings of the 31st International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 32, pp. 754–762. PMLR (2014)
29. Huynh Thi Thanh, B., Van Cuong, L., Thang, T.B., Long, N.H.: Ensemble Multifactorial Evolution With Biased Skill-Factor Inheritance for Many-Task Optimization. IEEE Transactions on Evolutionary Computation **27**(6), 1735–1749 (Dec 2023). https://doi.org/10.1109/TEVC.2022.3227120
30. Liang, J., Qiao, K., Yuan, M., Yu, K., Qu, B., Ge, S., Li, Y., Chen, G.: Evolutionary multi-task optimization for parameters extraction of photovoltaic models. Energy Conversion and Management **207**, 112509 (Mar 2020). https://doi.org/10.1016/j.enconman.2020.112509
31. Liu, J., Li, P., Wang, G., Zha, Y., Peng, J., Xu, G.: A multitasking electric power dispatch approach with multi-objective multifactorial optimization algorithm. IEEE access : practical innovations, open solutions **8**, 155902–155911 (2020). https://doi.org/10.1109/ACCESS.2020.3018484
32. Liu, P., Guo, Z., Yu, H., Linghu, H., Li, Y., Hou, Y., Ge, H., Zhang, Q.: A preliminary study of multi-task map-elites with knowledge transfer for robotic arm

design. In: 2022 IEEE Congress on Evolutionary Computation (CEC). pp. 1–8 (2022). https://doi.org/10.1109/CEC55065.2022.9870374
33. Lundberg, S.M., Erion, G., Chen, H., DeGrave, A., Prutkin, J.M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., Lee, S.I.: From local explanations to global understanding with explainable AI for trees. Nature Machine Intelligence **2**(1), 56–67 (2020). https://doi.org/10.1038/s42256-019-0138-9
34. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. In: Advances in Neural Information Processing Systems. vol. 30 (2017)
35. Mann, H.B., Whitney, D.R.: On a test of whether one of two random variables is stochastically larger than the other. The Annals of Mathematical Statistics **18**(1), 50–60 (1947). https://doi.org/10.1214/aoms/1177730491
36. Michie, D., Chambers, R.A.: BOXES: An experiment in adaptive control. In: Dale, E., Michie, D. (eds.) Machine Intelligence 2, pp. 137–152. Oliver and Boyd, Edinburgh (1968)
37. Mouret, J.B.: pymap_elites: Python reference implementation of MAP-Elites and Multi-Task MAP-Elites. https://github.com/resibots/pymap_elites
38. Mouret, J., Clune, J.: Illuminating search spaces by mapping elites. CoRR **abs/1504.04909** (2015), http://arxiv.org/abs/1504.04909
39. Mouret, J.B., Maguire, G.: Quality diversity for multi-task optimization. In: Proceedings of the 2020 Genetic and Evolutionary Computation Conference. pp. 121–129. ACM, Cancún Mexico (Jun 2020). https://doi.org/10.1145/3377930.3390203
40. Osaba, E., Martinez, A.D., Ser, J.D.: Evolutionary multitask optimization: a methodological overview, challenges and future research directions (2021), https://arxiv.org/abs/2102.02558
41. Pearce, M., Branke, J.: Continuous multi-task bayesian optimisation with correlation. European Journal of Operational Research **270**(3), 1074–1085 (2018). https://doi.org/10.1016/j.ejor.2018.03.017
42. Rendell, L., Boyd, R., Cownden, D., Enquist, M., Eriksson, K., Feldman, M.W., Fogarty, L., Ghirlanda, S., Lillicrap, T., Laland, K.N.: Why copy others? Insights from the social learning strategies tournament. Science **328**(5975), 208–213 (2010). https://doi.org/10.1126/science.1184719
43. Sagarna, R., Ong, Y.S.: Concurrently searching branches in software tests generation through multitask evolution. In: 2016 IEEE Symposium Series on Computational Intelligence (SSCI). pp. 1–8 (2016). https://doi.org/10.1109/SSCI.2016.7850040
44. Samarakoon, S.M.B.P., Muthugala, M.A.V.J., Elara, M.R.: Metaheuristic based navigation of a reconfigurable robot through narrow spaces with shape changing ability. Expert Systems with Applications **201**, 117060 (2022). https://doi.org/10.1016/j.eswa.2022.117060
45. Shahriari, B., Swersky, K., Wang, Z., Adams, R.P., de Freitas, N.: Taking the human out of the loop: A review of bayesian optimization. Proceedings of the IEEE **104**(1), 148–175 (2016). https://doi.org/10.1109/JPROC.2015.2494218
46. Triebold, C., Yaman, A.: Evolving generalist controllers to handle a wide range of morphological variations. In: Proceedings of the Genetic and Evolutionary Computation Conference. pp. 1137–1145 (2024)
47. Vassiliades, V., Mouret, J.B.: Discovering the elite hypervolume by leveraging interspecies correlation. In: Proceedings of the Genetic and Evolutionary Computation Conference. pp. 149–156. Gecco '18, Association for Computing Machinery, New York, NY, USA (2018). https://doi.org/10.1145/3205455.3205602

48. Vassiliades, V., Chatzilygeroudis, K., Mouret, J.B.: Using centroidal voronoi tessellations to scale up the multi-dimensional archive of phenotypic elites algorithm. IEEE Transactions on Evolutionary Computation **PP**, 1–1 (Aug 2017). https://doi.org/10.1109/TEVC.2017.2735550
49. Wang, C., Liu, J., Wu, K., Wu, Z.: Solving Multitask Optimization Problems With Adaptive Knowledge Transfer via Anomaly Detection. IEEE Transactions on Evolutionary Computation **26**(2), 304–318 (Apr 2022). https://doi.org/10.1109/TEVC.2021.3068157
50. Yaman, A., Bredeche, N., Çaylak, O., Leibo, J.Z., Lee, S.W.: Meta-control of social learning strategies. PLoS computational biology **18**(2), e1009882 (2022)
51. Yaman, A., Iacca, G.: Distributed embodied evolution over networks. Applied Soft Computing **101**, 106993 (2021)
52. Yim, M., Shen, W.M., Salemi, B., Rus, D., Moll, M., Lipson, H., Klavins, E., Chirikjian, G.S.: Modular self-reconfigurable robot systems. IEEE Robotics & Automation Magazine **14**(1), 43–52 (2007). https://doi.org/10.1109/MRA.2007.339623
53. Zhao, H., Ning, X., Liu, X., Wang, C., Liu, J.: What makes evolutionary multitask optimization better: A comprehensive survey. Applied Soft Computing **145**, 110545 (2023). https://doi.org/10.1016/j.asoc.2023.110545
54. Zhong, J., Feng, L., Cai, W., Ong, Y.S.: Multifactorial genetic programming for symbolic regression problems. IEEE Transactions on Systems, Man, and Cybernetics: Systems **50**(11), 4492–4505 (2020). https://doi.org/10.1109/TSMC.2018.2853719

A Node Coverage Guarantees Under Random Sampling

If we draw at each evaluation step a random node and perform an action, we may ask how many evaluations are required so that we visit each of the ν nodes at least once with high certainty.

This problem can be framed as a *coupon collector's problem* [21]. Writing M for the number of draws, the expected number of steps needed to visit all ν nodes at least once is

$$\mathbb{E}[M] = \nu H_\nu = \nu \left(\ln \nu + \gamma + O\left(\frac{1}{\nu}\right) \right),$$

where H_ν is the ν-th harmonic number and $\gamma \approx 0.57721$ is the Euler-Mascheroni constant.

Beyond the expectation, one may also ask for a high-probability guarantee. Using the asymptotic limit for the coupon collector's completion time, the probability that all nodes have been visited by draw

$$M = \nu(\ln \nu + c)$$

converges to

$$\mathbb{P}\big(M \leq \nu(\ln \nu + c)\big) \approx e^{-e^{-c}}.$$

To achieve at least 99% probability of having visited all nodes, we solve for c

$$e^{-e^{-c}} = p$$

$$\ln\Big(e^{-e^{-c}}\Big) = \ln(p)$$

$$-e^{-c} = \ln(p)$$

$$\text{For } p = 0.99: \quad c = -\ln\big(-\ln(0.99)\big) \approx 4.60015 \approx \ln(100).$$

Since each node is selected independently with probability $\frac{1}{\nu}$ at every draw, the visit count of any particular node over M draws follows a Binomial distribution,

$$X_i \sim \text{Binomial}\big(M, \tfrac{1}{\nu}\big)\,,$$

from which the expected number of visits per node is

$$\mathbb{E}[X_i] = \frac{M}{\nu}.$$

We evaluate four environments with differing numbers of distinct tasks and a fixed amount of draws (number of evaluations) $M = 1{,}000{,}000$:

Table 2: Coupon collector bounds for the four benchmark environments with $M = 1{,}000{,}000$ total evaluations.

Environment	ν	$\mathbb{E}[M]$	$M_{0.99}$	$\mathbb{E}[X_i]$
Archery / Arm / Cartpole	5 000	45 472	65 612	200
Hexapod	2 000	16 356	24 412	500

B MONET Algorithm

Algorithm 2 and Algorithm 3 detail the two learning subroutines invoked in Algorithm 1.

Individual Learning. When individual learning is selected, the current elite at node $\boldsymbol{\tau}_i$ is mutated independently of any neighbor. If no solution exists at the node, a random solution is drawn from $\mathcal{U}(p_{\min}, p_{\max})$ and evaluated. Otherwise, Gaussian mutation with standard deviation σ_m scaled by the parameter range is applied. The offspring replaces the current elite only if its fitness is at least as high.

Algorithm 2 Individual Learning

Require: Task $\boldsymbol{\tau}$, archive $\mathcal{A}$, bounds $p_{\min}, p_{\max}$, mutation std σ_m
Ensure: Updated $\mathcal{A}[\boldsymbol{\tau}]$

```
if A[τ] = ∅ then                                   ▷ initialize empty node
    θ ~ U(p_min, p_max)                            ▷ random solution
    A[τ] ← (θ, f(θ, τ))
    return
end if
θ ← A[τ].solution                                  ▷ current elite
ε ~ N(0, σ_m(p_max − p_min))                       ▷ Gaussian perturbation
y ← clip(θ + ε, p_min, p_max)                      ▷ mutant offspring
f_y ← f(y, τ)
if f_y ≥ A[τ].fitness then                         ▷ elitist replacement
    A[τ] ← (y, f_y)
end if
```

Social Learning. When social learning is selected, the current elite is recombined with a solution drawn from a neighboring node. If the current node is empty, the neighbor's solution is copied as an initial seed (or a random solution is used when no neighbor has a solution). Recombination is performed via Simulated Binary Crossover (SBX) [16] with distribution index $\eta_c = 10$ [15]. As with individual learning, the offspring replaces the elite only upon improvement.

Algorithm 3 Social Learning

Require: Task $\boldsymbol{\tau}$, neighbor task $\boldsymbol{\tau}_j$, archive $\mathcal{A}$, bounds $p_{\min}, p_{\max}$
Ensure: Updated $\mathcal{A}[\boldsymbol{\tau}]$

```
if A[τ] = ∅ then                                   ▷ initialize empty node
    if A[τ_j] ≠ ∅ then
        θ ← A[τ_j].solution                        ▷ copy from neighbor
    else
        θ ~ U(p_min, p_max)                        ▷ random fallback
    end if
    A[τ] ← (θ, f(θ, τ))
    return
end if
if A[τ_j] = ∅ then
    return                                         ▷ no valid neighbor solution
end if
y ← SBX(A[τ].solution, A[τ_j].solution, η_c = 10)  ▷ crossover
y ← clip(y, p_min, p_max)
f_y ← f(y, τ)
if f_y ≥ A[τ].fitness then                         ▷ elitist replacement
    A[τ] ← (y, f_y)
end if
```

C Closed-Form Task Fitness Definitions

This appendix gives the explicit fitness expressions for the four benchmarks.

C.1 Archery

With initial velocity vector

$$\mathbf{v} = v_0 \begin{bmatrix} -\sin(\text{yaw}) \\ \cos(\text{yaw})\cos(\text{pitch}) \\ \cos(\text{yaw})\sin(\text{pitch}) \end{bmatrix}, \quad v_0 = 70\,\text{m/s},$$

the target-plane hit time is $t_{\text{hit}} = D/v_y$ whenever $v_y > 0$; if $v_y \leq 0$ the shot misses and $f = 0$. Taking $g = 9.8\,\text{m/s}^2$ and wind $W \in [-10, 10]\,\text{m/s}^2$ along x, the miss coordinates on the target plane ($y = D$) are

$$m_x = \tfrac{1}{2}\,W\,t_{\text{hit}}^2 + v_x\,t_{\text{hit}}, \qquad m_z = -\tfrac{1}{2}\,g\,t_{\text{hit}}^2 + v_z\,t_{\text{hit}}, \qquad r = \sqrt{m_x^2 + m_z^2}.$$

Fitness is computed on a 10-ring target of ring width 0.061 m:

$$f = \frac{\max\bigl(0,\ 10 - \lfloor r/0.061 \rfloor\bigr)}{10} \in \{0, 0.1, \ldots, 1.0\}.$$

C.2 Arm

With end-effector position $\mathbf{p}_{\text{ee}} \in \mathbb{R}^2$ and a fixed target $\mathbf{p}_{\text{tar}} = (0.5, 0.5)$,

$$f = \exp(-\|\mathbf{p}_{\text{ee}} - \mathbf{p}_{\text{tar}}\|_2) \in (0, 1].$$

The joint commands are $\mathbf{q} = (\boldsymbol{\theta} - \frac{1}{2}) \cdot 2\pi\,\alpha_{\max}/10$ where $\boldsymbol{\theta} \in [0, 1]^{10}$ is the solution vector and $\alpha_{\max}$ is the task-dependent maximum joint-angle fraction; each of the 10 links has length $L/10$ with L the task-dependent total arm length. $\mathbf{p}_{\text{ee}}$ is the end-effector position returned by forward kinematics.

C.3 Cartpole

The task uses the `CartPole-v1` dynamics from OpenAI Gym with the pole mass m_p and pole length L replaced by the per-task values; the default cart mass m_c is retained. With pole angle ϕ, cart position x_c, control force $F \in \{-10, +10\}$ N, and gravity $g = 9.8\,\text{m/s}^2$,

$$\ddot{\phi} = \frac{g\sin\phi + \cos\phi\left(-F - m_p L\dot{\phi}^2\sin\phi\right)/(m_c + m_p)}{L\left(\frac{4}{3} - \frac{m_p\cos^2\phi}{m_c + m_p}\right)}, \qquad \ddot{x}_c = \frac{F + m_p L\left(\dot{\phi}^2\sin\phi - \ddot{\phi}\cos\phi\right)}{m_c + m_p}.$$

A solution is evaluated over $n_{\text{rollouts}} = 10$ rollouts with distinct Gym seeds. Each rollout runs until the pole leaves the balanced region ($|\phi| \geq 12^\circ$ or $|x_c| \geq 2.4\,\text{m}$) or until the step cap $t_{\max} = 1000$ is reached. Writing $t_{\text{balanced}}^{(k)}$ for the number of surviving steps on rollout k, the reported fitness is the arithmetic mean

$$f = \frac{1}{n_{\text{rollouts}}} \sum_{k=1}^{n_{\text{rollouts}}} \min\bigl(t_{\max},\ t_{\text{balanced}}^{(k)}\bigr) \in [0, t_{\max}].$$

C.4 Hexapod

With gait parameters $\boldsymbol{\theta} \in [0,1]^{36}$ and morphology $\boldsymbol{\tau} \in \mathcal{T}$ instantiated through a per-task URDF, a PyBullet simulation is run for up to 3 s of wall-clock simulator time. The episode terminates early if any body-axis angle (roll, pitch, yaw) exceeds $\pi/8$ or if the lateral deviation satisfies $|y| > 0.5\,\mathrm{m}$. Writing $p_x(t)$ for the forward position of the center of mass and $T \leq 3\,\mathrm{s}$ for the time of termination or episode end,

$$f(\boldsymbol{\theta}, \boldsymbol{\tau}) = p_x(T),$$

reported in metres. The controller returns $f = 0$ if any entry of $\boldsymbol{\theta}$ lies outside $[0,1]$.

D Detailed Sensitivity Analysis

This appendix provides per-domain SHAP beeswarm plots and the SPOT-tuned hyperparameter configurations that complement the global sensitivity analysis.

D.1 Per-Domain SHAP Analysis

Figures 6 and 7 show the SHAP beeswarm plots for the archery and arm domains, respectively. Each point represents one MONET run; its horizontal position indicates the SHAP value (contribution to the predicted metric), and its color encodes the feature value (red = high, blue = low).

In both domains, p_{ind} exhibits the strongest effect: high values of p_{ind} (more individual learning) consistently decrease predicted fitness, corroborating the global finding that social learning is the primary driver of performance. The remaining parameters show smaller, more task-specific effects.

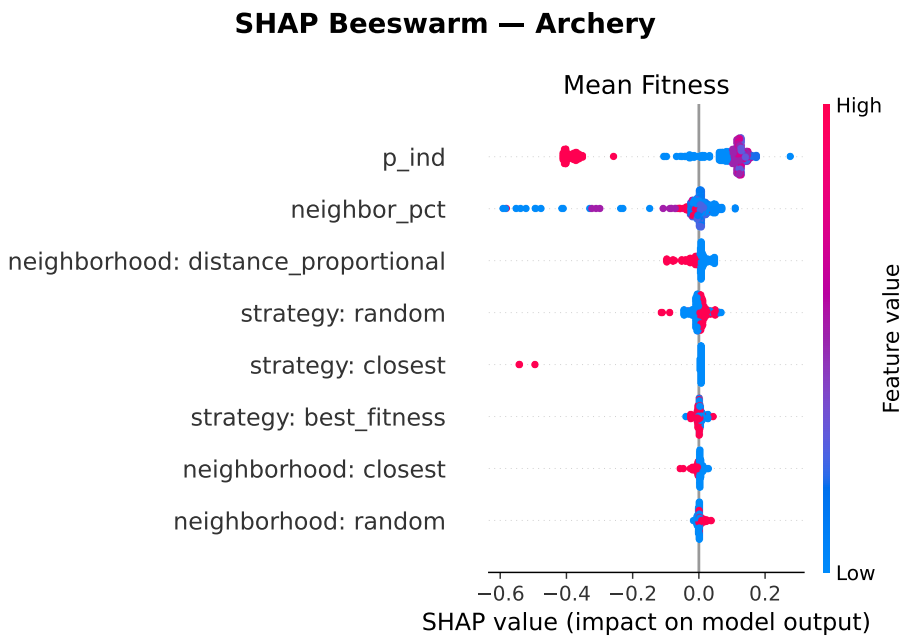


Fig. 6: SHAP beeswarm plot for the archery domain. Each row is a hyperparameter; each dot is a MONET run. Color indicates the feature value (red = high, blue = low); horizontal position indicates the SHAP value.

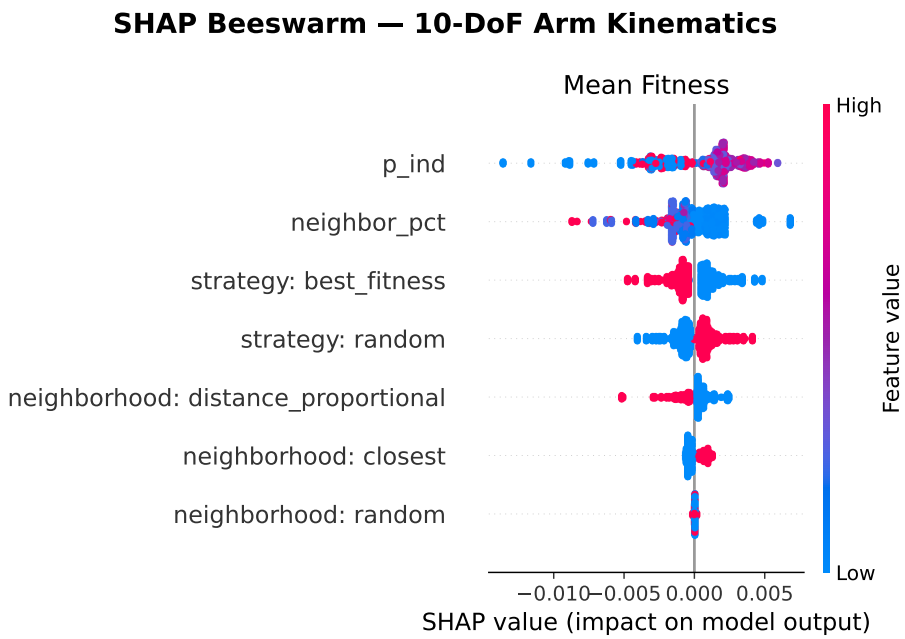


Fig. 7: SHAP beeswarm plot for the arm domain. Panels as in Figure 6.

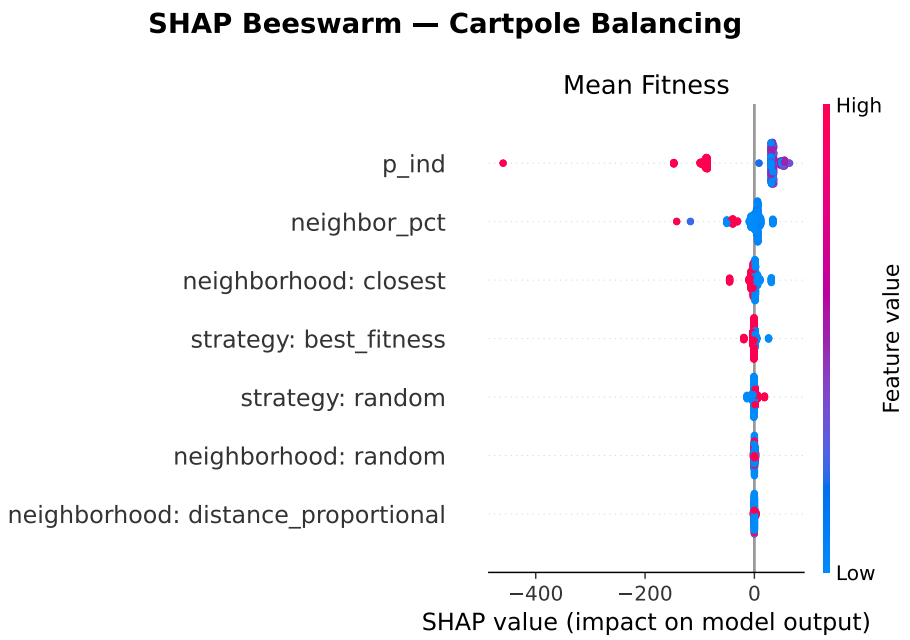


Fig. 8: SHAP beeswarm plot for the cartpole domain. Panels as in Figure 6.

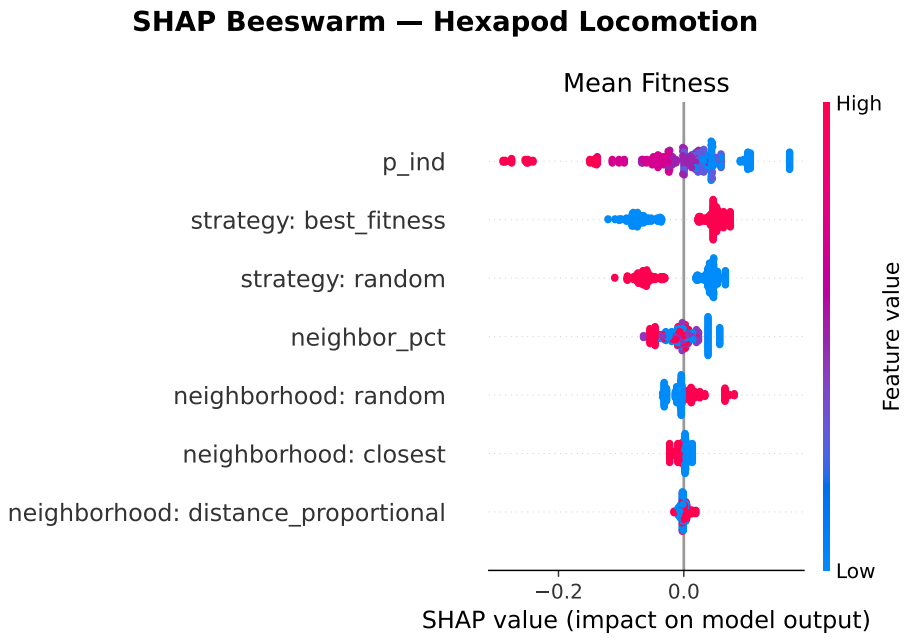


Fig. 9: SHAP beeswarm plot for the hexapod domain. Panels as in Figure 6.

D.2 SPOT-Tuned Configurations

Table 3 lists the per-domain configurations found by the SPOT hyperparameter tuner.

Table 3: SPOT-tuned MONET configurations per domain.

Domain	strategy	neighborhood	p_{ind}	neighbor_pct.
Archery	best_fitness	closest	0.0	2%
Arm	random	closest	0.3	1%
Cartpole	best_fitness	random	0.5	1%
Hexapod	best_fitness	random	0.0	1%

E MONET Hyperparameter Descriptions

Table 4 summarizes the four hyperparameters varied in the sensitivity analysis. All other algorithmic parameters (mutation operator, crossover operator, initialization strategy) are held fixed throughout. See Algorithm 1 for the points at which each parameter enters the loop.

Table 4: MONET hyperparameters and their roles.

Parameter	Values	Description
p_{ind}	0, 0.3, 0.5, 0.7, 1	Probability that a given evaluation step performs *individual learning* (mutation of the focal task's own elite) rather than *social learning* (crossover with a neighbor's elite). At $p_{\text{ind}} = 0$ only social learning occurs; at $p_{\text{ind}} = 1$ only individual learning occurs.
strategy	random, best_fitness	Neighbor selection strategy during social learning. `random`: uniform random selection among all neighbors. `best_fitness`: deterministically select the neighbor with the highest current fitness.
neighborhood	random, closest, distance_prop.	Method for constructing the neighbor set of each task. `closest`: the N nearest tasks by Euclidean distance in task space. `random`: N tasks sampled uniformly at random (sorted by similarity after sampling). `distance_proportional`: N tasks sampled *without replacement* with probability proportional to task-space similarity $s(\boldsymbol{\tau}_i, \boldsymbol{\tau}_j) = 1/(1 + \|\boldsymbol{\tau}_i - \boldsymbol{\tau}_j\|)$; this softly biases the neighborhood toward nearby tasks while retaining a chance of including distant ones.
neighbor_pct.	1%, 5%, 10%	Fraction of the total task set used as the neighborhood size $N = \lceil \texttt{neighbor_percentage} \cdot \lvert\mathcal{V}\rvert \rceil$. For instance, with 5 000 tasks and 5%, each task has $N =$ 250 neighbors. Controls graph sparsity: lower values yield sparser graphs with more localized knowledge transfer.

F Hyperparameter Configuration Ranking

Table 5 lists the ten highest-ranked MONET hyperparameter configurations (across the 2–4 domains on which they were run) by combined z-score, computed as the mean of the within-domain z-scores of final mean fitness and AUC. Table 6 shows the five worst-ranked configurations for comparison. Only configurations evaluated on at least two domains are included in the ranking. The counts $n_{\text{Arm}}, n_{\text{Archery}}, n_{\text{Cartpole}}, n_{\text{Hexapod}}$ report the number of deduplicated seed runs per domain.

Two patterns are clear from Table 5: (i) the `closest` neighborhood dominates the top of the ranking, and (ii) small neighborhood sizes ($\leq 2\%$ of the task set) are preferred. Poorly-performing configurations (Table 6) combine the `distance_proportional` neighborhood with either very high p_{ind} (near pure individual learning) or very large neighborhoods, both of which disrupt local knowledge transfer.

Table 5: Top 10 MONET configurations by combined z-score ($n_{\text{domains}} \geq 2$).

strategy	neighborhood	nbr.%	p_{ind}	n_{Arm}	n_{Arc}	n_{Cart}	n_{Hex}	z_{FF}	z_{AUC}	combined
best_fitness	closest	1.5	0.30	20	20	0	0	+0.58	+1.01	**+0.80**
best_fitness	closest	1.0	0.30	22	22	0	5	+0.60	+0.96	+0.78
best_fitness	closest	2.0	0.30	20	20	0	0	+0.55	+0.99	+0.77
best_fitness	closest	2.0	0.50	20	39	5	0	+0.62	+0.90	+0.76
best_fitness	closest	1.5	0.70	5	20	0	0	+0.62	+0.81	+0.72
best_fitness	closest	2.0	0.70	5	20	0	0	+0.62	+0.82	+0.72
random	closest	2.0	0.00	20	19	4	0	+0.66	+0.75	+0.71
random	closest	1.5	0.00	20	19	5	0	+0.66	+0.75	+0.70
best_fitness	closest	1.0	0.50	40	41	5	5	+0.56	+0.82	+0.69
random	closest	1.0	0.00	40	40	5	3	+0.64	+0.73	+0.68

Table 6: Bottom 5 MONET configurations by combined z-score ($n_{\text{domains}} \geq 2$).

strategy	neighborhood	nbr.%	p_{ind}	n_{Arm}	n_{Arc}	n_{Cart}	n_{Hex}	z_{FF}	z_{AUC}	combined
best_fitness	distance_prop.	5.0	1.00	20	20	0	2	−1.22	−1.86	−1.54
best_fitness	distance_prop.	10.0	0.00	20	20	0	16	−2.51	−0.65	−1.58
random	closest	34.3	0.98	2	1	1	0	−1.19	−1.97	−1.58
best_fitness	distance_prop.	5.0	0.00	20	20	0	3	−3.00	−0.93	−1.96
best_fitness	distance_prop.	41.5	0.99	2	2	2	0	−1.97	−2.23	**−2.10**

G Code Availability

Table 7 lists the reference papers and public code repositories for MONET and the three baselines.

Table 7: Reference papers and public code repositories for MONET and the baseline algorithms.

Algorithm	**Paper**	**Code**
MONET (ours)	This paper	[25]
MAP-Elites	[14, 38]	[37]
MT-ME	[39]	[37]
PT-ME	[4]	[5]